

Dossier d'Architecture et de Conception

de la Base de Données (DCBD)

Projet : mobiTrace (Traçabilité, Réemploi et Affectation Circulaire des Équipements)

Moteur : PostgreSQL 17 + PostgREST

Version : 2.0

1. Modèle Conceptuel et Logique de Données (MCD / MLD)

L'architecture relationnelle de mobiTrace est construite autour de l'équipement physique (le mobilier) qui fait le lien entre une définition abstraite (le gabarit) et une affectation spatiale/administrative (lieu et structure).

Entités et Cardinalités

- Un LIEU héberge de 0 à N STRUCTURES.
- Un LIEU contient de 0 à N MOBILIERS.
- Une STRUCTURE (SAGES) appartient à 1 et 1 seul LIEU (clé étrangère lieu_id).
- Une STRUCTURE possède de 0 à N MOBILIERS.
- Un GABARIT (Modèle catalogue) définit de 0 à N MOBILIERS.
- Un MOBILIER est défini par 1 et 1 seul GABARIT, est affecté à 1 et 1 seule STRUCTURE et se trouve dans 1 et 1 seul LIEU.

2. Dictionnaire de Données

Ce dictionnaire liste l'ensemble des tables, vues et schémas, ainsi que la structure de leurs champs.

2.1 Table mobiliers (Inventaire du parc)

C'est la table centrale du système, contenant les équipements physiques.

Champ	Type SQL	Contraintes / Valeurs	Description
uuid	UUID	PRIMARY KEY, Auto (gen_random_uuid())	Identifiant technique unique global.
id_metier	VARCHAR(20)	UNIQUE	N° de série généré par trigger (ex : MOB-000123).
gabarit_id	INTEGER	FK → gabarits(id)	Référence au modèle du catalogue.
lieu_id	INTEGER	FK → lieux(id)	Localisation physique actuelle.
code_sages	VARCHAR(10)	FK → structures(code_sages)	Affectation administrative du matériel.
statut	ENUM statut_mobilier	Défaut : en_service	État : en_service, en_maintenance, dispo_reemploi, au_rebut.
remarques	TEXT	Nullable	Observations saisies par l'agent.
date_saisie	TIMESTAMP WITH TIME ZONE	Défaut : NOW()	Date et heure de l'intégration au parc, conservée avec le fuseau horaire.

2.2 Table gabarits (Catalogue National)

Référentiel des modèles d'équipements respectant la Charte de Nommage.

Champ	Type SQL	Contraintes / Valeurs	Description
id	SERIAL	PRIMARY KEY	Identifiant technique.

reference_catalogue	VARCHAR(50)	UNIQUE, Not Null	Code normalisé (ex : BUR-001).
categorie	TEXT	Nullable	Famille (Bureau, Assise, Rangement, Autre).
nom_descriptif	TEXT	Not Null	Appellation standard (ex : "Chaise empilable").
caracteristiques	JSONB	Défaut : {}	Attributs dynamiques (voir règles JSON ci-dessous).
photo_base64	TEXT	Nullable	Vignette illustrative encodée en Base64.

Règles du champ JSONB (caracteristiques) :

— Format des clés : snake_case sans accents (ex : largeur_cm).

— Booléens : stockés en chaînes de texte ("true", "false").

— Clés recommandées : reference_marche, reference_ugap, annee_acquisition (ou null).

2.3 Tables structures et lieux (Référentiel organisationnel)

Table structures

Champ	Type SQL	Contraintes	Description
code_sages	VARCHAR(10)	PRIMARY KEY	Identifiant métier du service.
libelle	TEXT	Not Null	Nom complet du service.
lieu_id	INTEGER	FK → lieux(id)	Rattachement au bâtiment.

Table lieux

Champ	Type SQL	Contraintes	Description
id	SERIAL	PRIMARY KEY	Identifiant technique.
nom	TEXT	Not Null	Nom du bâtiment ou de la localisation.

2.4 Table utilisateurs (Gestion des accès)

Champ	Type SQL	Contraintes / Valeurs	Description
id	SERIAL	PRIMARY KEY	Identifiant technique.
email	TEXT	UNIQUE, Not Null	Identifiant de connexion (ex : admin@gouv.fr).
nom_complet	TEXT	Not Null	Nom d'affichage. Initialisé avec la partie locale de l'email (avant le @).
mot_de_passe_hash	TEXT	Not Null	Hash Bcrypt généré par pgcrypto (gen_salt('bf')).
role	TEXT	CHECK (agent administrateur lecteur) Défaut : agent	Rôle applicatif déterminant les droits dans la matrice RBAC.

Accès : la table utilisateurs est révoquée pour les rôles agent et lecteur (REVOKE ALL). Seul l'administrateur y a accès via le panel d'administration.

2.5 Table audit_logs (Journal de traçabilité)

Remplie exclusivement et automatiquement par les Triggers du système.

Champ	Type SQL	Contraintes	Description
id	SERIAL	PRIMARY KEY	Identifiant technique.
date_action	TIMESTAMP WITH TIME ZONE	Défaut : NOW()	Horodatage de l'action avec fuseau horaire.
utilisateur	VARCHAR(255)		Email de l'agent ayant réalisé l'action (extrait du JWT). Vaut 'Système' si l'action provient d'un processus interne.
action	VARCHAR(50)		Nature de l'opération : CRÉATION, MODIFICATION ou SUPPRESSION.
id_metier	VARCHAR(50)		Cible de l'action (ex : MOB-123456, BUR-002, ou un email utilisateur).
details	TEXT		Différentiel précis des champs modifiés (OLD → NEW).

Mécanismes d'immuabilité

La table `audit_logs` est protégée contre toute altération par deux mécanismes cumulatifs opérant au niveau du moteur SQL :

— `RULE no_update_audit` : règle PostgreSQL qui bloque physiquement toute instruction `UPDATE` sur la table en la remplaçant par `NOTHING`. Aucun client, quel que soit son rôle, ne peut modifier une entrée existante.

— `Trigger trig_protect_audit_delete` (`BEFORE DELETE`) : la fonction `protect_audit_delete()` vérifie l'identité de l'exécutant (`current_user`) et n'autorise la suppression que si celui-ci est le superutilisateur 'postgres'. Toute autre tentative lève une exception explicite.

Ces deux mécanismes garantissent l'intégrité légale du journal d'audit, y compris contre une action directe d'un administrateur applicatif via un client lourd (pgAdmin, psql).

2.6 Table parametres (Paramétrage global) — NOUVEAU

Cette table remplace l'ancien fichier `config.ini`. Elle stocke les paramètres d'identité de la direction utilisant `mobiTrace` et est lue au démarrage de l'application front-end via l'endpoint `PostgREST /parametres`.

Champ	Type SQL	Contraintes	Description
cle	VARCHAR(50)	PRIMARY KEY	Identifiant du paramètre (ex : <code>nom_administration</code>).
valeur	TEXT	Not Null	Valeur associée au paramètre.

Valeurs insérées par défaut à l'installation :

— `nom_administration` : "MINISTÈRE DE L'ÉCONOMIE ET DES FINANCES"

— `nom_direction` : "Direction Générale des Finances Publiques"

Droits : lecture accordée à `divagil`, `agent`, `administrateur`, `lecteur`. Écriture réservée au rôle `administrateur`.

2.7 Schéma auth (Infrastructure JWT) — NOUVEAU

L'ensemble des primitives cryptographiques est isolé dans un schéma dédié (`auth`), séparé du schéma public exposé par

PostgREST. Ce schéma n'est jamais accessible directement depuis l'API.

Table `auth.secrets`

Stocke les secrets sensibles sous forme de paires clé/valeur.

Champ	Type SQL	Contraintes	Description
key	TEXT	PRIMARY KEY	Identifiant du secret.
value	TEXT	Not Null	Valeur du secret.

À l'installation, le `postinst` génère une valeur aléatoire de 48 caractères pour la clé `'jwt_secret'` via `/dev/urandom`. La valeur est injectée par substitution de la balise `__JWT_SECRET__` avant exécution du schéma SQL. Elle n'est jamais écrite en clair dans un fichier de configuration.

Fonction `auth.url_encode(data bytea) → text`

Encode un bloc de données binaires en Base64 URL-safe (RFC 7515 — JSON Web Signature). Remplace les caractères `+`, `/`, `=` et les sauts de ligne par `-`, `_` et rien.

Fonction `auth.sign_jwt(payload json, secret text) → text`

Implémentation complète de la signature JWT HS256 en PL/pgSQL pur, sans dépendance à une bibliothèque externe.

Algorithme :

- Encodage de l'en-tête fixe `{"alg":"HS256","typ":"JWT"}` en Base64 URL.
- Encodage du payload JSON fourni en Base64 URL.
- Calcul du HMAC-SHA256 de `(header.payload)` avec le secret via `pgcrypto hmac()`.
- Concaténation : `header_b64 || '.' || payload_b64 || '.' || signature_b64`.

Appelée exclusivement par `public.login()` après vérification du mot de passe. Non exposée via PostgREST.

2.8 Vue `vue_mobiliers_recherche` — NOUVEAU

Vue de jointure principale consommée par l'API PostgREST pour toutes les opérations de consultation du parc. Elle dénormalise en une seule requête les quatre tables métier centrales.

Jointures :

```
mobiliers
  ⌘ gabarits ON mobiliers.gabarit_id = gabarits.id
  ⌘ structures ON mobiliers.code_sages = structures.code_sages
  ⌘ lieux ON mobiliers.lieu_id = lieux.id
```

Champs supplémentaires exposés (par rapport à la table `mobiliers` brute) :

Champ exposé	Source	Description
<code>gabarit_nom</code>	<code>gabarits.nom_descriptif</code>	Désignation standard du modèle.
<code>gabarit_categorie</code>	<code>gabarits.categorie</code>	Famille du modèle (Bureau, Assise...).
<code>gabarit_json_txt</code>	<code>gabarits.caracteristiques::text</code>	Attributs techniques en texte brut (consommé par

gabarit_photo	gabarits.photo_base64	l'index Trigram).
structure_libelle	structures.libelle	Vignette du modèle encodée en Base64.
lieu_nom	lieux.nom	Nom complet du service affectataire.
		Nom du bâtiment de localisation.

*Seuls les mobiliers ayant un gabarit, une structure et un lieu valides apparaissent dans cette vue (jointure INNER).
Droits : SELECT accordé à divagil, agent, administrateur, lecteur.*

3. Modèle Physique de Données (MPD) et Optimisations

mobiTrace repose sur une architecture "Database-Centric". La base de données ne se contente pas de stocker, elle exécute la logique métier.

3.1 Indexation haute performance

Pour garantir une recherche globale instantanée même avec des centaines de milliers d'équipements, des index spécifiques sont implémentés :

Index	Table	Type	Colonnes / Expression	Rôle
idx_mobiliers_gabarit	mobiliers	B-tree	gabarit_id	Accélère les filtres par modèle de catalogue.
idx_mobiliers_ua	mobiliers	B-tree	code_sages	Accélère les filtres par service affectataire.
idx_mobiliers_statut	mobiliers	B-tree	statut	Accélère les filtres par état du matériel.
idx_mobiliers_lieu	mobiliers	B-tree	lieu_id	Accélère les filtres par localisation.
idx_mobiliers_search_trgm	mobiliers	GIN (pg_trgm)	(id_metier ' ' COALESCE(remarques,rapide sans ILIKE '%...%'. ''))	Recherche textuelle ultra-rapide sans ILIKE '%...%'. ''))
idx_gabarits_caract	gabarits	GIN	caracteristiques	Recherche dans les attributs JSONB dynamiques à la vitesse d'une colonne standard.

3.2 Génération automatique et atomicité (ACID)

Afin d'éviter tout conflit lors d'imports massifs ou d'accès concurrents :

Identifiant Métier (Trigger BEFORE INSERT)

La fonction set_mob_id() et la séquence seq_mobilier_id garantissent que chaque équipement reçoit un numéro unique au format MOB-XXXXXX juste avant l'insertion. Les numéros sont générés par nextval() atomique, éliminant les conditions de course (Race Conditions) même lors d'un Bulk Insert concurrent.

Fonctions de session et d'administration

Fonction	Retour	Accès	Description
public.login(email, password)	jwt_token	divagil	Vérifie le mot de passe via crypt() (Bcrypt). Signe le JWT HS256 (durée 8h) et pose deux cookies HttpOnly : trace_token avec Path=/ (session principale) et un cookie vide avec Path=/api Max-Age=0 (efface un éventuel résidu sur /api).
public.logout()	void	Tous les rôles	Invalide la session en émettant deux cookies Set-Cookie Max-Age=0 sur Path=/ et Path=/api.
public.me()	json	Tous les rôles	Retourne {email, role} depuis les claims JWT courants. Lève l'exception PT401 si aucun jeton valide n'est présent.
public.creer_utilisateur(_email, void _password, _role)		administrateur	Crée un compte avec hash Bcrypt. Le sel est généré à chaque appel. REVOKE sur PUBLIC.
public.reinitialiser_mdp(_email void , _new_password)		administrateur	Réinitialise le hash Bcrypt. Le sel est régénéré à chaque appel. REVOKE sur PUBLIC.

Fonctions métier avancées

Fonction	Retour	Accès	Description
public.get_filtres_disponibles(p_li jsonb eu_id, p_code_sages, p_gabarit_id)		Tous les rôles	Moteur de facettes. Retourne {lieux, structures, gabarits} calculé en tenant compte des deux autres filtres actifs (filtrage croisé). Les listes vides sont normalisées en [].
public.caracteristiques_txt(g gabarits)	text	Tous les rôles	Computed column PostgREST. Expose le champ JSONB caracteristiques en TEXT pour l'index Trigram. Déclarée IMMUTABLE.

3.3 Traçabilité inaltérable (Audit Trail)

La journalisation n'est pas gérée par l'application web mais exclusivement par des Triggers SQL. Même une modification directe en base via un client lourd (pgAdmin, psql) sera historisée dans audit_logs.

Inventaire complet des triggers

Trigger	Table cible	Moment / Événement	Fonction appelée
trig_audit_mobiliers	mobiliers	AFTER INSERT / UPDATE / DELETE	log_mobilier_action()
trig_audit_gabarits	gabarits	AFTER INSERT / UPDATE / DELETE	log_gabarit_action()
trig_audit_structures	structures	AFTER INSERT / UPDATE / DELETE	log_admin_action()
trig_audit_utilisateurs	utilisateurs	AFTER INSERT / UPDATE / DELETE	log_admin_action()
trig_audit_lieux	lieux	AFTER INSERT / UPDATE / DELETE	log_admin_action()
trig_check_mob_update	mobiliers	BEFORE UPDATE	check_mobilier_update_rights()
trig_protect_audit_delete	audit_logs	BEFORE DELETE	protect_audit_delete()

Détail des fonctions de journalisation

- `log_mobilier_action()` : trace les opérations sur l'inventaire physique. En cas de MODIFICATION, calcule le différentiel OLD vs NEW et ne trace que les champs réellement modifiés parmi : `code_sages`, `lieu_id`, `statut`, `gabarit_id`, `remarques`. Si aucun de ces champs ne change, aucune ligne n'est insérée (pas de bruit dans les logs).
- `log_gabarit_action()` : trace les évolutions du catalogue (création, modification de référence/désignation/catégorie/attributs JSON/photo). Préfixe "CATALOGUE :" dans le champ details.
- `log_admin_action()` : trace les opérations d'administration sur structures, lieux et utilisateurs. Préfixe "ADMINISTRATION :" dans le champ details.

Trigger de contrôle des droits métier

`check_mobilier_update_rights()` — BEFORE UPDATE sur mobiliers

Lit le rôle de l'utilisateur courant depuis les claims JWT (`request.jwt.claims`). Si le champ `gabarit_id` est modifié et que le rôle n'est pas 'administrateur', l'opération est bloquée avec l'exception :

```
ACTION_BLOQUEE: Seul un administrateur peut modifier le modèle (gabarit) d'un équipement.
```

En cas d'absence de jeton (accès système direct), le rôle 'agent' est appliqué par défaut (principe de moindre privilège).

3.4 Sécurité (RLS / RBAC et JWT)

L'API PostgREST se repose intégralement sur les mécanismes natifs de PostgreSQL. Aucune logique de sécurité n'est implémentée dans le code applicatif front-end.

Authentification RPC

La fonction `public.login()` réalise les étapes suivantes :

- Recherche l'utilisateur par email et vérifie le mot de passe via `crypt()` (Bcrypt, pgcrypto).
- En cas de succès, construit un payload JWT : `{role, user_id, email, exp = now + 8h}`.
- Signe le token via `auth.sign_jwt()` (HS256, secret lu dans `auth.secrets`).
- Pose deux cookies `HttpOnly Secure SameSite=Strict` simultanément :

```
trace_token=<jwt> ; Path=/ -> cookie de session principal
trace_token= ; Path=/api ; Max-Age=0 -> efface un résidu eventuel sur /api
```

Le token JWT n'est jamais exposé dans le corps de la réponse HTTP. Il transite uniquement via les cookies.

Matrice des droits RBAC

Quatre rôles sont définis. Le rôle `divagil` est le compte de connexion PostgREST ; il hérite dynamiquement du rôle applicatif porté par le JWT.

Objet	lecteur	agent	administrateur
mobiliers	SELECT	SELECT, UPDATE	SELECT, INSERT, UPDATE, DELETE
gabaris	SELECT	SELECT	SELECT, INSERT, UPDATE, DELETE

lieux	SELECT	SELECT	SELECT, INSERT, UPDATE, DELETE
structures	SELECT	SELECT	SELECT, INSERT, UPDATE, DELETE
parametres	SELECT	SELECT	SELECT, UPDATE
vue_mobiliers_recherche	SELECT	SELECT	SELECT
utilisateurs	AUCUN	AUCUN	SELECT, INSERT, UPDATE, DELETE
audit_logs	AUCUN	AUCUN	SELECT uniquement
Séquences (public.*)	—	—	ALL

Points de sécurité notables

— La table utilisateurs est explicitement révoquée pour agent et lecteur (REVOKE ALL). L'accès à la liste des comptes est réservé à l'administrateur.

— La table audit_logs est révoquée pour tous sauf SELECT pour l'administrateur. Aucun rôle applicatif ne peut écrire ou supprimer directement dans ce journal (uniquement via SECURITY DEFINER).

— Les fonctions sensibles (creer_utilisateur, reinitialiser_mdp) sont déclarées SECURITY DEFINER, révoquées sur PUBLIC et accordées au seul rôle administrateur.

— Les fonctions de session (login, logout, me) sont SECURITY DEFINER pour qu'elles accèdent à auth.secrets et utilisateurs même depuis un rôle sans droits directs sur ces tables.